Knowledge Representation

Lecture 3: Queries, Actions with Qualifications

dr Anna Maria Radzikowska

Warsaw University of Technology
Faculty of Mathematics and Information Science
Building MiNI PW, room 504

E-mail: Anna.Radzikowska@pw.edu.pl

Warsaw 2026

Outline

Action Language $\mathcal{AR}$: Syntax

Statements

- ***Value statement:***
  α ***after*** $A_1, \ldots, A_n$
  observable α ***after*** $A_1, \ldots, A_n$.
- ***Effect statement:***
  ACTION ***causes*** α ***if*** π
- ***Releases statement:***
  ACTION ***releases*** f ***if*** π
- ***Constraint statement:***
  always α
- ***Fluent specification statement:***
  noninertial f.

Action language $\mathcal{AR}$: Semantics

Structure

A ***structure*** for a language $\mathcal{L}$ of the class $\mathcal{AR}$ is a triple $S = (\Sigma, \sigma_0, Res)$, where

- Σ is a set of states
- $\sigma_0 \in \Sigma$ is the initial state
- $Res : \mathcal{A}c \times \Sigma \to 2^{\Sigma}$ is a transition function.

Model

A structure $S = (\Sigma, \sigma_0, Res)$ is a ***model*** of an action domain D iff

(M1) Σ is the set of all states for D (all constraints are satisfied)

(M2) every value statement and every observation statement is true in S

(M3) for every $A \in \mathcal{A}c$ and for every $\sigma \in \Sigma$, $Res(A, \sigma)$ is the set of all states $\sigma' \in \Sigma$ for which the sets $New(A, \sigma, \sigma')$ are minimal (wrt set inclusion), where $New(A, \sigma, \sigma')$ is the set of literals $\overline{f}$ that hold in σ' and

- f is inertial and $\sigma(f) \neq \sigma'(f)$, or
- there is a statement A ***releases*** f ***if*** π in D such that $\sigma \models \pi$.

Queries

Queries

Value queries

necessary α ***after*** $A_1, \ldots, A_n$ ***from*** π
possibly α ***after*** $A_1, \ldots, A_n$ ***from*** π

The 1st query states that α **always** holds after performing the sequence $A_1, \ldots, A_n$ of actions from any state satisfying π.
The 2nd query says that α **sometimes** holds after executing $A_1, \ldots, A_n$ from any state satisfying π.
When the option ***from*** π is omitted, these queries refer to the initial state.

Queries (cont.)

Executability queries

necessary executable $A_1, \ldots, A_n$ ***from*** π
possibly executable $A_1, \ldots, A_n$ ***from*** π.

Intuitively, the 1^{st} (resp. the 2^{nd}) query states that from every state satisfying π the sequence $(A_1, \ldots, A_n)$ of actions is **always** (resp. **sometimes**) executable.

Again, if the phrase ***from*** π is omitted, then the initial state is to be taken into account.

Queries (cont.)

Accessibility queries

necessary accessible γ ***from*** π
possibly accessible γ ***from*** π.

These queries state that the goal γ is **always** (resp. **sometimes**) achieved from any state satisfying π.

INTUITIVELY:

There exists a sequence $(A_1, \ldots, A_n)$, $n \geqslant 0$, *of actions, executable from any state satisfying* π *which always (resp. ever) leads to states satisfying* γ. *Different sequences of actions, starting in different states where* π *holds, may lead to states satisfying the goal* γ.

As before, if ***from*** π is omitted, then it refers to the initial state.

Queries (cont.)

Goal query

goal γ ***from*** π

This query returns the shortes sequence $(A_1, \ldots, A_n)$ of actions, executable from any state satisfying π (if such a sequence exists!), which leads to states satisfying the goal condition γ.

If the phrase "***from*** π" is omitted, then the goal γ is to be achieved from the initial state.

Satisfiability

Let D be an action domain and let Q be a query of the language $\mathcal{AR}$. A query is a ***consequence of*** D, in symbols $D \approx Q$, iff

necessary α after $A_1, \ldots, A_n$ from π

$D \approx Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D, for every state $\sigma \in \Sigma$ and for **every** mapping Ψ_s, $\sigma \models \pi$ implies $\Psi_S((A_1, \ldots, A_n), \sigma) \models \alpha$.

necessary α after $A_1, \ldots, a_n$

$D \approx Q$ iff for every model $s = (\Sigma, \sigma_0, Res)$ of D, and for **every** mapping Ψ_S, it holds $\Psi_S((a_1, \ldots, A_n), \sigma_0) \models \alpha$.

Satisfiability (cont.)

Q : *possibly* α *after* $A_1, \ldots, A_n$ *from* π

$D \models\!\!\approx Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D, for every state $\sigma \in \Sigma$, and for **some** mapping Ψ_s, if $\sigma \models \pi$, then $\Psi_S((A_1, \ldots, A_n), \sigma) \models \alpha$.

Q : *possibly* α *after* $A_1, \ldots, A_n$

$D \models\!\!\approx Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D and for **some** mapping Ψ_s, it holds $\Psi_S((A_1, \ldots, A_n), \sigma_0) \models \alpha$.

Example: Tossing a coin

Example 3.1

Let an action domain be as follows:

$$\textbf{\textit{initially}}\ heads;$$
$$\textsc{Toss}\ \textbf{\textit{releases}}\ heads.$$

Then

$$D \mid\!\approx \textbf{\textit{possibly}}\ heads\ \textbf{\textit{after}}\ \textsc{Toss}$$
$$D \mid\!\approx \textbf{\textit{possibly}}\ \neg heads\ \textbf{\textit{after}}\ \textsc{Toss}$$
$$D \not\mid\!\approx \textbf{\textit{necessary}}\ \neg heads\ \textbf{\textit{after}}\ \textsc{Toss}$$
$$D \not\mid\!\approx \textbf{\textit{necessary}}\ head\ \textbf{\textit{after}}\ \textsc{Toss}.$$

Satisfiability (cont.)

Q: *necessary executable* $A_1, \ldots, A_n$ *from* π

$D \approx\!\!\models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D, for every state $\sigma \in \Sigma$, and for **every** mapping Ψ_S, if $\sigma \models \pi$, then $\Psi_S((A_1, \ldots, A_n), \sigma)$ is defined.

Q: *necessary executable* $A_1, \ldots, A_n$

$D \approx\!\!\models Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D and for **every** mapping Ψ_S, $\Psi_S((A_1, \ldots, A_n), \sigma_0)$ is defined.

Satisfiability (cont.)

Q: *possibly executable* $A_1, \ldots, A_n$ *from* π

$D \models\!\!\approx Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D, for every state $\sigma \in \Sigma$, and for **some** mapping Ψ_S, if $\sigma \models \pi$, then $\Psi_S((A_1, \ldots, A_n), \sigma)$ is defined.

Q: *possibly executable* $A_1, \ldots, A_n$

$D \models\!\!\approx Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D and for **some** mapping Ψ_S, $\Psi_S((A_1, \ldots, A_n), \sigma_0)$ is defined.

Satisfiability (cont.)

Q: *necessary accessible* γ *from* π

$D \models\!\!\approx Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D, for every state $\sigma \in \Sigma$, and for **every** mappings Ψ_S, if $\sigma \models \pi$, then there is some sequence $(A_1, \ldots, A_k)$, $k \geqslant 0$, of actions such that

- $\Psi_S((A_1, \ldots, A_k), \sigma)$ is defined;
- $\Psi_S((A_1, \ldots, A_k), \sigma) \models \gamma$.

Q: *necessary accessible* γ

$D \models\!\!\approx Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D and for **every** mappings Ψ_S, there is some sequence $(A_1, \ldots, A_k)$, $k \geqslant 0$, of actions such that

- $\Psi_S((A_1, \ldots, A_k), \sigma_0)$ is defined;
- $\Psi_S((A_1, \ldots, A_k), \sigma_0) \models \gamma$.

Satisfiability (cont.)

Q: *possibly accessible* γ *from* π

$D \models\!\!\approx Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D, for every state $\sigma \in \Sigma$, and for **some** mappings Ψ_S, if $\sigma \models \pi$, then there is some sequence $(A_1, \ldots, A_k)$, $k \geqslant 0$, of actions such that

- $\Psi_S((A_1, \ldots, A_k), \sigma)$ is defined;
- $\Psi_S((A_1, \ldots, A_k), \sigma) \models \gamma$.

Q: *possibly accessible* γ

$D \models\!\!\approx Q$ iff for every model $S = (\Sigma, \sigma_0, Res)$ of D and for **some** mappings Ψ_S, there is some sequence $(A_1, \ldots, A_k)$, $k \geqslant 0$, of actions such that

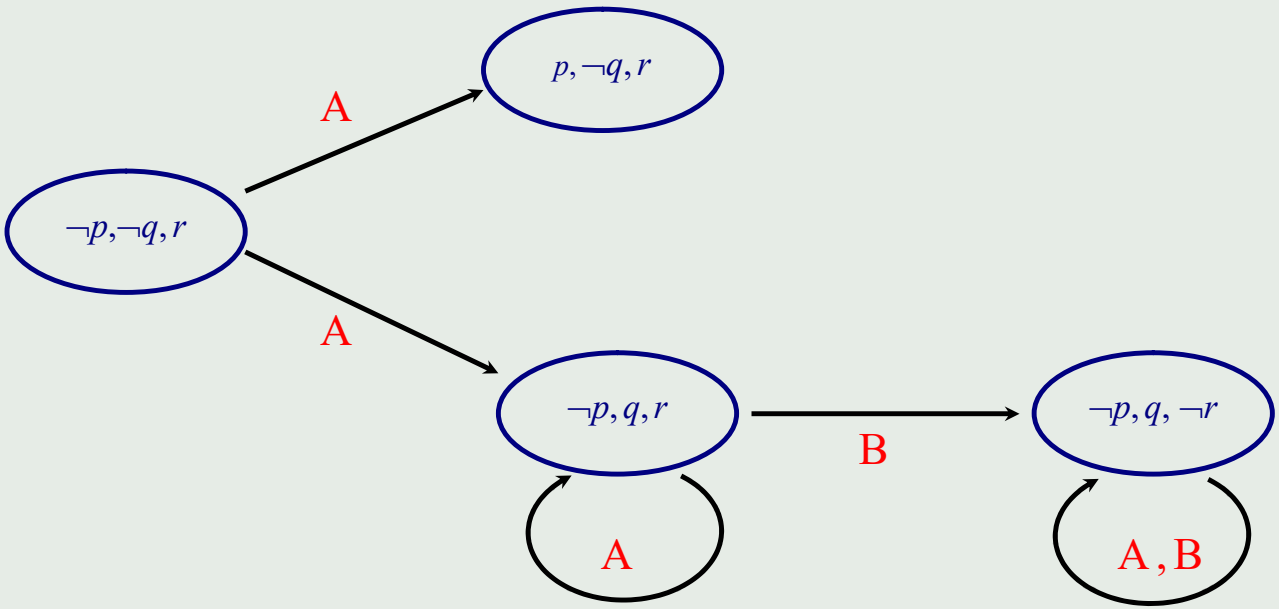
- $\Psi_S((A_1, \ldots, A_k), \sigma_0)$ is defined;
- $\Psi_S((A_1, \ldots, A_k), \sigma_0) \models \gamma$.

Example

Example 3.2

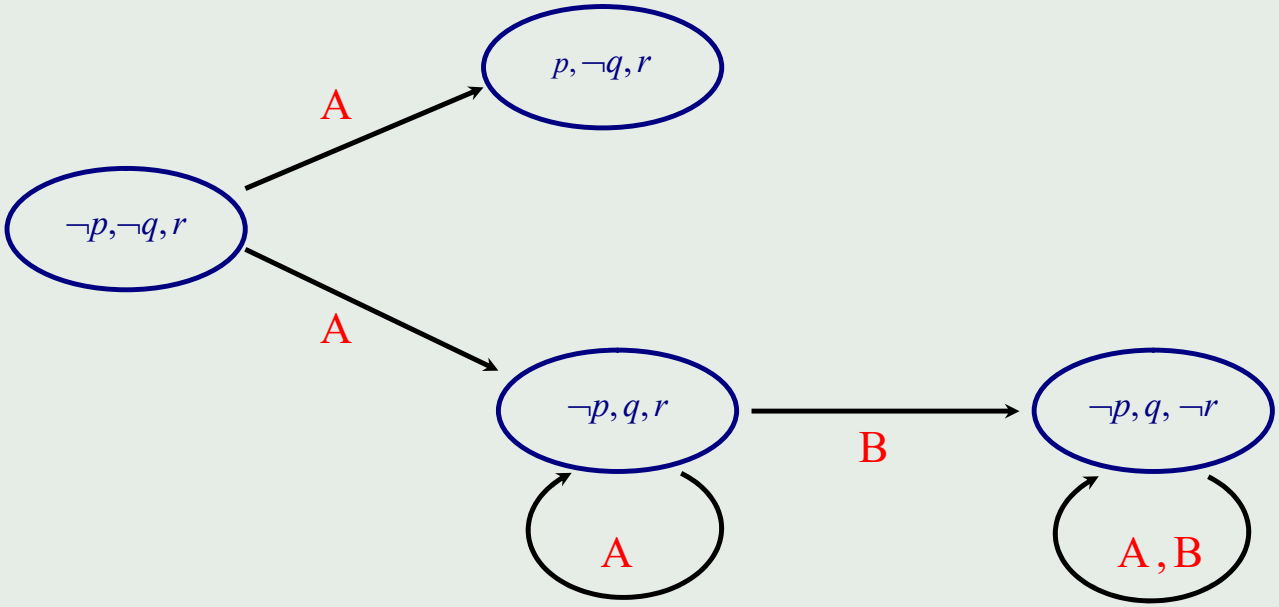
Consider an action domain:

$$\textsc{ActionA} \textbf{ \textit{causes} } p \vee q;$$
$$\textsc{ActionB} \textbf{ \textit{causes} } \neg r;$$
$$\textbf{\textit{impossible}} \ \textsc{ActionB} \ \textbf{\textit{if}} \ \neg q.$$



Example (cont.)

Example 3.2 (cont.)



Note:

$D \not\models$ ***necessary accessible*** $\neg r$ ***from*** $\neg p$.

$D \models$ ***possibly accessible*** $\neg r$ ***from*** $\neg p$.

Satisfiability (cont.)

Q: *goal* γ *from* π

$D \approx\!\models Q$ iff there is some sequence $(A_1, \ldots, A_n)$ of action such that for every models $S = (\Sigma, \sigma_0, Res)$ of D, for every state $\sigma \in \Sigma$, and for **every** mapping Ψ_S, if $\sigma \models \pi$, then

- $\Psi_S((A_1, \ldots, A_n), \sigma)$ is defined;
- $\Psi_S((A_1, \ldots, A_n), \sigma) \models \gamma$.

The sequence $(A_1, \ldots, A_n)$, if exists, is called an ***answer for*** the query Q wrt D.

Q: *goal* γ

$D \approx\!\models Q$ iff there is some sequence $(A_1, \ldots, A_n)$ of action such that for every models $S = (\Sigma, \sigma_0, Res)$ of D and for **every** mapping Ψ_S,

- $\Psi_S((A_1, \ldots, A_n), \sigma_0)$ is defined;
- $\Psi_S((A_1, \ldots, A_n), \sigma_0) \models \gamma$.

Actions with Qualifications

Qualification problem

The qualification problem concerns conditions under which the performance of an action is possible, it proceeds successfully and leads to expected results. Since all preconditions of actions are usually immense, it is usually unreasonable (if ever possible) to explicitly enumerate and check all of possibilities (even most unlikely).

Introductory example (Lin & Reiter, 1994)

Example 3.3

Consider the following scenario:

> *In an ancient kingdom there are two blocks, A and B. Either block may be painted yellow, but by order of the emperor at most one of the blocks is permitted to be yellow at a time. Initially the first block is yellow. A painter tried to paint the second one yellow.*

Representation in $\mathcal{AR}$:

$$\textbf{\textit{always}}\ \neg yellowA \vee \neg yellowB;$$
$$\textbf{\textit{initially}}\ yellowA;$$
$$\textsc{PaintA}\ \textbf{\textit{causes}}\ yellowA;$$
$$\textsc{PaintB}\ \textbf{\textit{causes}}\ yellowB.$$

Introductory example (cont.)

Example 3.3 (cont.)

Here $\Sigma = \{\sigma_0, \sigma_1, \sigma_2\}$ where

$$\sigma_0 = \{yellowA, \neg yellowB\} \quad \sigma_1 = \{\neg yellowA, \neg yellowB\}.$$
$$\sigma_2 = \{\neg yellowA, yellowB\}$$

Note that

$$Res_0(\textsc{PaintB}, \sigma_0) = Res(\textsc{PaintB}, \sigma_0) = \{\sigma_2\}.$$

So painting the block B changes the color of the block A!

Example: Enticing Fred

Example 3.3

Consider the following action domain:

$$\textbf{\textit{initially}}\ loaded \wedge walking;$$
$$\textbf{\textit{always}}\ walking \rightarrow alive;$$
$$\textsc{Shoot}\ \textbf{\textit{causes}}\ \neg loaded;$$
$$\textsc{Shoot}\ \textbf{\textit{causes}}\ \neg alive\ \textbf{\textit{if}}\ loaded;$$
$$\textsc{Entice}\ \textbf{\textit{causes}}\ walking.$$

Consider the following states

$$\sigma_0 = \{alive, loaded, walking\}$$
$$\sigma_2 = \{\neg alive, \neg loaded, \neg walking\}.$$

Note that

$$Res(\textsc{Shoot}, \sigma_0) = \{\sigma_1\}.$$

Example: Enticing Fred (cont.)

Example 3.3 (cont.)

Furthermore,

$$\sigma_2 = \{\neg alive, loaded, \neg walking\}$$
$$\sigma_3 = \{alive, \neg loaded, walking\}$$
$$\sigma_4 = \{alive, loaded, walking\}.$$

In $\mathcal{AR}$ we get:

$$Res_0(\textsc{Entice}, \sigma_2) = \{\sigma_3, \sigma_4\}$$
$$New(\textsc{Entice}, \sigma_2, \sigma_3) = \{alive, \neg loaded, walking\}$$
$$* \quad New(\textsc{Entice}, \sigma_2, \sigma_4) = \{alive, walking\}$$
$$Res(\textsc{Entice}, \sigma_2) = \{\sigma_4\}.$$

So enticing a dead turkey makes him miraculously alive!!!

The reasoning method of $\mathcal{AR}$ is **NOT** adequate to represent this scenario. Another language is needed!

Basic assumptions

- Inertia law.
- Complete information about all actions and all fluents.
- Nondeterminism is allowed.
- **Whenever any indirect precondition of an action fails, the action is unexecutable**.
- Action can be unexecutable is some states.

Action language $\mathcal{AQ}$

To represent this class of dynamic systems we will use action languages of the class $\mathcal{AQ}$.

Action Language $\mathcal{AQ}$

Syntax

Identical as in $\mathcal{AR}$.

Semantics

Let D be an action domain in a language $\mathcal{L}$ of the class $\mathcal{AQ}$. A ***structure*** for a language $\mathcal{L}$ is a triple $S = (\Sigma, \sigma_0, Res)$ such that

- Σ is a set of states;
- $\sigma_0 \in \Sigma$ is the initial state;
- $Res : \mathcal{A}c \times \Sigma \rightarrow 2^{\Sigma}$ is a transition function.

Semantics of $\mathcal{AQ}$ (cont.)

Notation

- D^- : the action domain obtained from D by removing all constraint statements.
- $Mod(D^-)$: the set of all models $S^- = (\Sigma^-, \sigma_0, Res^-)$ of D^- (in the sense of $\mathcal{AR}$) where σ_0 satisfies all constraints in D.

Definition 3.1

Le D be an action domain and let $(\Sigma^-, \sigma_0, Res^-) \in Mod(D^-)$. A ***model*** of D is a structure (Σ, σ_0, Res) such that

- $\Sigma \subseteq \Sigma^-$ is the set of all states satisfying all constraints in D;
- for every $A \in \mathcal{A}c$ and for every $\sigma \in \Sigma$, $Res(A, \sigma) = Res^-(A, \sigma) \cap \Sigma$.

Introductory example (cont.)

Example 3.3 (cont.)

$$\textbf{\textit{always}}\ \neg yellowA \vee \neg yellowB;$$
$$\textbf{\textit{initially}}\ yellowA;$$
$$\textsc{PaintA}\ \textbf{\textit{causes}}\ yellowA;$$
$$\textsc{PaintB}\ \textbf{\textit{causes}}\ yellowB.$$

Now $\Sigma^{-} = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$ where

$$\sigma_0 = \{yellowA, \neg yellowB\} \qquad \sigma_2 = \{\neg yellowA, yellowB\}$$
$$\sigma_1 = \{\neg yellowA, \neg yellowB\} \qquad \sigma_3 = \{yellowA, yellowB\}.$$

$$Res_0(\textsc{PaintB}, \sigma_0) = \{\sigma_2, \sigma_3\}$$
$$New(\textsc{PaintB}, \sigma_0, \sigma_2) = \{\neg yellowA, yellowB\}$$
$$* \quad New(\textsc{PaintB}, \sigma_0, \sigma_3) = \{yellowB\}$$
$$Res^{-}(\textsc{PaintB}, \sigma_0) = \{\sigma_3\}$$
$$Res(\textsc{PaintB}, \sigma_0) = \emptyset.$$

Introductory example (cont.)

Example 3.3 (cont.)

$$\sigma_0 = \{yellowA, \neg yellowB\} \qquad \sigma_2 = \{\neg yellowA, yellowB\}$$
$$\sigma_1 = \{\neg yellowA, \neg yellowB\} \qquad \sigma_3 = \{yellowA, yellowB\}.$$

Analogously:

$$Res_0^-(\textsc{PaintA}, \sigma_2) = \{\sigma_0, \sigma_3\}$$
$$New(\textsc{PaintA}, \sigma_2, \sigma_0) = \{yellowA, \neg yellowB\}$$
$$* \quad New(\textsc{PaintA}, \sigma_2, \sigma_3) = \{yellowA\}$$
$$Res^-(\textsc{PaintA}, \sigma_2) = \{\sigma_3\}$$
$$Res(\textsc{PaintA}, \sigma_2) = \emptyset.$$

Introductory example (cont.)

Example 3.3 (cont.)

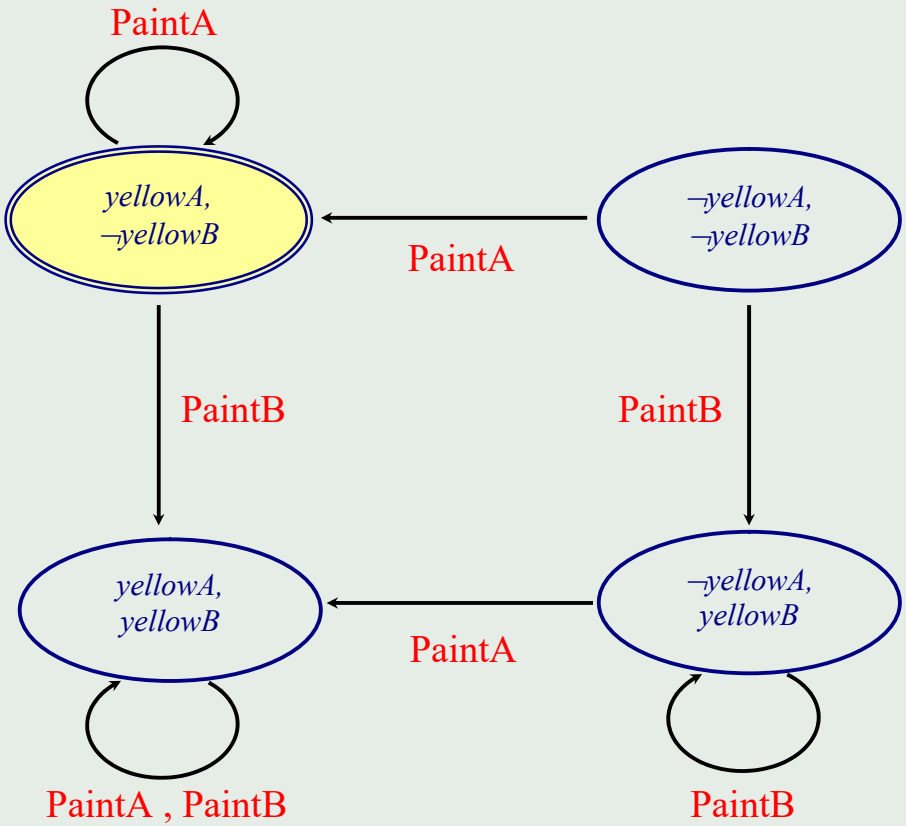


Fig.3.1 : Model of D^{-}

Example 3.3 (cont.)

Example: Enticing Fred (cont.)

Example 3.3 (cont.)

initially $loaded \wedge walking$;
always $walking \rightarrow alive$;
SHOOT ***causes*** $\neg loaded$;
SHOOT ***causes*** $\neg alive$ ***if*** $loaded$;
ENTICE ***causes*** $walking$.

$\Sigma^- = \{\sigma_0, \ldots, \sigma_7\}$ where

$\sigma_0 = \{alive, loaded, walking\}$
$\sigma_1 = \{alive, \neg loaded, walking\}$
$\sigma_2 = \{\neg alive, loaded, walking\}$
$\sigma_3 = \{\neg alive, \neg loaded, walking\}$
$\sigma_4 = \{alive, loaded, \neg walking\}$
$\sigma_5 = \{alive, \neg loaded, \neg walking\}$
$\sigma_6 = \{\neg alive, loaded, \neg walking\}$
$\sigma_7 = \{\neg alive, \neg loaded, \neg walking\}$.

Example: Enticing Fred (cont.)

Example 3.3 (cont.)

$\sigma_0 = \{alive, loaded, walking\}$
$\sigma_1 = \{alive, \neg loaded, walking\}$
$\sigma_2 = \{\neg alive, loaded, walking\}$
$\sigma_3 = \{\neg alive, \neg loaded, walking\}$
$\sigma_4 = \{alive, loaded, \neg walking\}$
$\sigma_5 = \{alive, \neg loaded, \neg walking\}$
$\sigma_6 = \{\neg alive, loaded, \neg walking\}$
$\sigma_7 = \{\neg alive, \neg loaded, \neg walking\}.$

$$Res_0^-(\textsc{Entice}, \sigma_6) = \{\sigma_0, \sigma_1, \sigma_2, \sigma_3\}$$
$$New(\textsc{Entice}, \sigma_6, \sigma_0) = \{alive, walking\}$$
$$New(\textsc{Entice}, \sigma_6, \sigma_1) = \{alive, \neg loaded, walking\}$$
$$* \quad New(\textsc{Entice}, \sigma_6, \sigma_2) = \{walking\}$$
$$New(\textsc{Entice}, \sigma_6, \sigma_3) = \{\neg loaded, walking\}$$
$$Res^-(\textsc{Entice}, \sigma_6) = \{\sigma_2\}$$
$$Res(\textsc{Entice}, \sigma_6) = \emptyset.$$

Example: Enticing Fred (cont.)

Example 3.3 (cont.)

$\sigma_0 = \{alive, loaded, walking\}$
$\sigma_1 = \{alive, \neg loaded, walking\}$
$\sigma_2 = \{\neg alive, loaded, walking\}$
$\sigma_3 = \{\neg alive, \neg loaded, walking\}$
$\sigma_4 = \{alive, loaded, \neg walking\}$
$\sigma_5 = \{alive, \neg loaded, \neg walking\}$
$\sigma_6 = \{\neg alive, loaded, \neg walking\}$
$\sigma_7 = \{\neg alive, \neg loaded, \neg walking\}$.

Consider the shooting action in the initial state σ_0.

$Res_0^-(\text{SHOOT}, \sigma_0) = \{\sigma_3, \sigma_7\}$

* $New(\text{SHOOT}, \sigma_0, \sigma 3) = \{\neg alive, \neg loaded\}$
$New(\text{SHOOT}, \sigma_0, \sigma_7) = \{\neg alive, \neg loaded, \neg walking\}$

$Res^-(\text{SHOOT}, \sigma_0) = \{\sigma_3\}$
$Res(\text{SHOOT}, \sigma_0) = \emptyset$.

So it is impossible to shoot a walking!

Problem

Question

*Should we reject inadmissible states **before** or **after** minimization of changes?*

Answer

- If constraints influence only on actions' **ramifications** and do not influence on actions' qualifications, then inadmissible states are to be rejected **before** minimization of changes
- If constraints influence only on actions' **qualifications** and do not influence on actions' ramfications, then inadmissible states should be rejected **after** minimization of changes.

Another problem

Question

What should be done if a constraint works for ramifications wrt one action, and at the same time for qualifications wrt another one?

Possible solutions

Mixed reasoning methods: action languages $\mathcal{ARQ}$ and $\mathcal{AQR}$.

Action Language $\mathcal{ARQ}$

Syntax

The set of statements of $\mathcal{AR}$ is extended by an fluent preserving statement:

$$\boxed{\mathrm{A} \textbf{ preserves } f \textbf{ if } \pi}$$

Intuitively, in any state satisfying π, the action A, when performed, **does not** change the value of the fluent f

Action Language $\mathcal{ARQ}$ – semantics

Structure

A ***structure*** for a language $\mathcal{L}$ of the class $\mathcal{ARQ}$ is a triple $S = (\Sigma, \sigma_0, Res)$ where

- Σ is a set of states;
- $\sigma_0 \in \Sigma$ is the initial state;
- $Res : \mathcal{A}c \times \Sigma \to 2^{\Sigma}$ is a transition function.

Model

Let D be an action domain in $\mathcal{ARQ}$ and let D^- stand for the action domain obtained from D by removing all fluent preserving statements. Let $Mod(D^-)$ be the set of all models od D^- in the sense of $\mathcal{AR}$. A structure $S = (\Sigma, \sigma_0, Res)$ is a ***model*** of D iff for every action $A \in \mathcal{A}c$ and for every state $\sigma \in \Sigma$,

$$Res(A, \sigma) = \{\sigma' \in \Sigma : (A \textbf{ preserves } f \text{ if } \pi) \in D \wedge (\sigma \models \pi) \Rightarrow \sigma(f) = \sigma'(f)\}.$$

Example: Enticing Fred (cont.)

Example 3.3 (cont.)

initially $loaded \wedge walking$;
always $walking \rightarrow alive$;
SHOOT ***causes*** $\neg loaded$;
SHOOT ***causes*** $\neg alive$ ***if*** $loaded$;
ENTICE ***causes*** $walking$;
ENTICE ***preserves*** $alive$ ***if*** $loaded$.

Here $\Sigma = \{\sigma_0, \sigma_1, \ldots, \sigma_5\}$ where

$\sigma_0 = \{alive, loaded, walking\}$
$\sigma_1 = \{alive, loaded, \neg walking\}$
$\sigma_2 = \{\neg alive, loaded, \neg walking\}$
$\sigma_3 = \{alive, \neg loaded, walking\}$
$\sigma_4 = \{alive, \neg loaded, \neg walking\}$
$\sigma_3 = \{\neg alive, \neg loaded, \neg walking\}$

Example: Enticing Fred (cont.)

Example 3.3 (cont.)

$\sigma_0 = \{alive, loaded, walking\}$ $\quad$ $\sigma_3 = \{alive, \neg loaded, walking\}$
$\sigma_1 = \{alive, loaded, \neg walking\}$ $\quad$ $\sigma_4 = \{alive, \neg loaded, \neg walking\}$
$\sigma_2 = \{\neg alive, loaded, \neg walking\}$ $\quad$ $\sigma_3 = \{\neg alive, \neg loaded, \neg walking\}$

$$
\begin{array}{ll}
 & Res_0^-(\textsc{Entice}, \sigma_2) = \{\sigma_0, \sigma_3\} \\
* & New(\textsc{Entice}, \sigma_2, \sigma_0) = \{alive, walking\} \\
 & New(\textsc{Entice}, \sigma_2, \sigma_3) = \{alive, \neg loaded, walking\} \\
 & Res^-(\textsc{Entice}, \sigma_2) = \{\sigma_0\}
\end{array}
$$

Since $\sigma_2 \models \neg alive \wedge loaded$ and $\sigma_0 \models alive$, we get $\sigma_0 \notin Res(\textsc{Entice}, \sigma_2)$. Hence

$$Res(\textsc{Entice}, \sigma_2) = \emptyset.$$

Example: Enticing Fred (cont.)

Example 3.3 (cont.)

$$\sigma_0 = \{alive, loaded, walking\} \qquad \sigma_3 = \{alive, \neg loaded, walking\}$$
$$\sigma_1 = \{alive, loaded, \neg walking\} \qquad \sigma_4 = \{alive, \neg loaded, \neg walking\}$$
$$\sigma_2 = \{\neg alive, loaded, \neg walking\} \qquad \sigma_3 = \{\neg alive, \neg loaded, \neg walking\}$$

$$Res_0^-(\textsc{Shoot}, \sigma_0) = Res^-0(\textsc{Shoot}, \sigma_0) = \{\sigma_5\}.$$

Since there is no fluent preserving statement wrt $\textsc{Shoot}$ and $walking$, we obtain

$$Res(\textsc{Shoot}, \sigma_0) = \{\sigma_5\}.$$

Alternative solution

Representation in $\mathcal{AQ}$

We can use an $\mathcal{AQ}$ language: whenever constraints work for actions' ramifications, the respective ***releases*** statements are to be added.

Example 3.3 (cont.)

initially $loaded \wedge walking$;
always $walking \rightarrow alive$;
SHOOT ***causes*** $\neg loaded$;
SHOOT ***causes*** $\neg alive$ ***if*** $loaded$;
SHOOT ***releases*** $walking$ ***if*** $loaded$;
ENTICE ***causes*** $walking$.

Example: Enticing Fred (cont.)

Example 3.3 (cont.)

$\sigma_0 = \{alive, loaded, walking\}$ $\quad$ $\sigma_4 = \{alive, loaded, \neg walking\}$

$\sigma_1 = \{alive, \neg loaded, walking\}$ $\quad$ $\sigma_5 = \{alive, \neg loaded, \neg walking\}$

$\sigma_2 = \{\neg alive, loaded, walking\}$ $\quad$ $\sigma_6 = \{\neg alive, loaded, \neg walking\}$

$\sigma_3 = \{\neg alive, \neg loaded, walking\}$ $\quad$ $\sigma_7 = \{\neg alive, \neg loaded, \neg walking\}.$

$$Res_0^-(\textsc{Shoot}, \sigma_0) = \{\sigma_3, \sigma_7\}$$

$$* \quad New(\textsc{Shoot}, \sigma_0, \sigma_3) = \{\neg alive, \neg loaded, walking\}$$

$$* \quad New(\textsc{Shoot}, \sigma_0, \sigma_7) = \{\neg alive, \neg loaded, \neg walkikng\}$$

$$Res^-(\textsc{Shoot}, \sigma_0) = \{\sigma_3, \sigma_7\}$$

$$Res(\textsc{Shoot}, \sigma_0) = \{\sigma_7\}.$$

Thank you for your attention!

Questions are welcome.